LINUX NETWORK TROUBLESHOOTING

OSI 7계층 장애 진단·대응 실습 명령어 총정리

상황별 장애 재현 · 진단 · 복구 명령어 모음 (L1~L7 + 시스템/컨테이너/클라우드/보안/자동화 + 부록 H 변수분리 대응)

사용 방법

1. 0장의 network namespace 랩에서 장애를 일으키고 진단·복구를 반복해 익힙니다.
2. 실제 서버에서는 **읽기 전용 점검 명령**부터 사용하고, 설정 변경·방화벽 규칙 삽입은 반드시 영향도를 확인한 뒤 실행합니다.
3. 항상 **아래(L1)에서 위(L7)로** 확인하고, 이상 지점의 계층에서 원인을 좁힙니다.
4. **전역 상수:** 이름·IP·포트·경로는 0-1의 `lab.env` 변수($NS_C, $C_IP, $PORT 등)로 분리되어 있습니다. 값을 바꿀 때는 lab.env 만 수정하세요.
5. <꺾쇠> 부분은 환경에 맞게 바꿔 쓰세요. 일부 명령은 별도 패키지 설치가 필요합니다 (ethtool, tcpdump, mtr, iperf3, nmap, dnsutils 등).

주의 nmap·부하·DDoS 방어 실험·보안 점검 명령은 **본인이 관리하거나 허가받은 시스템에만** 사용하세요.

목차

0. 실습 랩 구성과 전역 상수 (network namespace)

1계층: 물리 계층 (Physical)

2계층: 데이터 링크 계층 (Data Link)

3계층: 네트워크 계층 (Network)

4계층: 전송 계층 (Transport)

5계층: 세션 계층 (Session)

6계층: 표현 계층 (Presentation)

7계층: 응용 계층 (Application)

부록 A. 시스템 자원 병목 (네트워크처럼 보이는 장애)

부록 B. 패킷 캡처 · 분석 종합

부록 C. 컨테이너 · 가상화 · 클라우드 환경

부록 D. 보안 관점 점검 (침입·공격 징후)

부록 E. 종합 진단 자동화 스크립트

부록 F. 증상 → 계층 빠른 판별표

부록 G. 계층별 핵심 명령어 한 장 요약

부록 H. 일방향 벡터·패턴·바이트 다리 (변수분리 대응 총정리)

0. 실습 랩 구성과 전역 상수 (network namespace)

운영 서버가 아닌 격리된 가상 환경에서 장애를 일으키고 진단·복구합니다. 이름·IP·포트·경로는 전부 lab.env 의 전역 상수로 분리했으므로, 환경이 달라지면 lab.env 의 값만 고치면 문서 전체 명령에 반영됩니다. 모든 명령은 root 권한 셸에서 실행하세요(sudo 는 변수를 넘기지 않으므로 sudo -i 로 root 셸을 연 뒤 사용).

0-1 전역 상수 정의 (lab.env) 와 불러오기

판단 readonly 라서 값 변경 불가(상수). 같은 셸에서 source 를 두 번 하면 오류가 나므로 새 터미널에서 다시 시작

```
cat > lab.env <<'EOF'
# ===== 랩 전역 상수: 이 파일 한 곳만 고치면 아래 모든 명령에 반영 =====
# --- 네임스페이스(가상 컴퓨터) 이름 ---
readonly NS_C=client           # 클라이언트 역할
readonly NS_S=server           # 서버 역할
# --- 가상 NIC(veth) 이름 ---
readonly C_IF=veth-c           # client 쪽 NIC
readonly S_IF=veth-s           # server 쪽 NIC
# --- IP 주소 ---
readonly NET=10.0.0.0/24       # 랩 네트워크 대역
readonly C_IP=10.0.0.1         # client IP
readonly S_IP=10.0.0.2         # server IP
readonly PREFIX=24             # 서브넷 마스크 길이(/24)
# --- 포트 ---
readonly PORT=8080             # 실습용 HTTP 포트
readonly TLS_PORT=8443         # 실습용 TLS 포트
# --- 경로 / 이름 ---
readonly WEBROOT=/tmp/www      # 웹 문서 폴더
readonly KEY=/tmp/k.pem        # 테스트용 개인키
readonly CRT=/tmp/c.pem        # 테스트용 인증서
readonly TLS_CN=test.local     # 테스트 인증서의 CN(호스트명)
EOF
source ./lab.env
```

0-2 랩 생성: client ↔ server 를 veth 케이블로 연결

판단 ping 2회 성공하면 정상. alias 는 source 이후, 다음 줄부터 사용 가능

```
ip netns add $NS_C; ip netns add $NS_S
ip link add $C_IF type veth peer name $S_IF
ip link set $C_IF netns $NS_C; ip link set $S_IF netns $NS_S
ip -n $NS_C addr add $C_IP/$PREFIX dev $C_IF
ip -n $NS_S addr add $S_IP/$PREFIX dev $S_IF
for n in $NS_C $NS_S; do ip -n $n link set lo up; done
ip -n $NS_C link set $C_IF up; ip -n $NS_S link set $S_IF up
alias C="ip netns exec $NS_C"     # 큰따옴표: 정의 시점에 변수가 값으로 치환됨
alias S="ip netns exec $NS_S"
C ping -c 2 $S_IP
```

0-3 간이 웹서버 기동 (HTTP)

판단 client 에서 ok 가 출력되면 L1~L7 전 구간 정상

```
mkdir -p $WEBROOT && echo ok > $WEBROOT/index.html
S python3 -m http.server $PORT --bind $S_IP --directory $WEBROOT &
C curl -s http://$S_IP:$PORT/
```

0-4 랩 초기화 / 정리

판단 웹서버(프로세스)를 먼저 종료한 뒤 네임스페이스 삭제

```
pkill -f "http.server $PORT"; pkill -f s_server
ip netns del $NS_C; ip netns del $NS_S
```

```
rm -f $KEY $CRT
```

1계층: 물리 계층 (Physical)

케이블, 전원, NIC, 스위치 포트, 광모듈(SFP), 링크 협상. 증상: 링크 다운, 속도·듀플렉스 불일치, CRC 에러 증가.

1-1 인터페이스 DOWN / 케이블 분리

판단 state DOWN, NO-CARRIER, Link detected: no → 케이블·포트·관리자 down

```
S ip link set $S_IF down            # 장애 발생
C ping -c 2 -W 1 $S_IP             # 실패
S ip -br link show $S_IF            # DOWN
ip -n $NS_C link show $C_IF        # NO-CARRIER
S ip link set $S_IF up              # 복구
```

1-2 링크 상태·속도·듀플렉스 점검

판단 Speed 10/100Mb 또는 Half 협상 = 케이블/포트/오토네고 문제

```
ip -br link
ethtool eth0 | egrep "Speed|Duplex|Auto-negotiation|Link detected"
ethtool -S eth0 | egrep -i "err|drop|crc|fifo|miss"
ethtool -i eth0                     # 드라이버/펌웨어 버전
ethtool -a eth0                     # 흐름제어(pause)
ethtool -k eth0 | egrep "tso|gso|gro|rx-checksum"   # 오프로드
cat /sys/class/net/eth0/{operstate,carrier,speed,duplex}
cat /sys/class/net/eth0/carrier_changes   # 링크 플래핑 횟수
```

1-3 링크 플래핑 · NIC 하드웨어 오류 로그

판단 link is Down/Up 반복 = 케이블 불량, SFP 불량, 스위치 포트 오류

```
dmesg -T | egrep -i "eth0|link is (up|down)|nic|firmware|tx timeout|reset adapter"
journalctl -k --since "1 hour ago" | grep -i link
watch -n1 'cat /sys/class/net/eth0/carrier_changes'
ip monitor link                      # 링크 상태 변화 실시간 감시
```

1-4 속도·듀플렉스 강제 설정 / 오토네고 복구

판단 양단 설정이 달라 Duplex mismatch 가 나면 late collision, CRC 증가

```
ethtool -s eth0 autoneg off speed 1000 duplex full   # 강제(양단 동일해야 함)
ethtool -s eth0 autoneg on                           # 복구
ethtool -r eth0                                      # 오토네고 재시작
```

1-5 광모듈(SFP/SFP+) · 케이블 진단

판단 RX power 가 낮으면 광케이블/모듈 오염·불량

```
ethtool -m eth0                     # DOM: 온도, TX/RX power, 파장
ethtool --cable-test eth0           # 지원 NIC 의 TDR 케이블 테스트(드라이버 의존)
lspci | grep -i ethernet; lspci -vv -s <BDF> | grep -i -E "LnkSta|LnkCap"   # PCIe 링크 폭/속도
lshw -class network -short
```

1-6 하드웨어 모니터링 (전원·온도·디스크)

판단 서버 자체 물리 장애 확인

```
ipmitool sdr; ipmitool sel list      # BMC 센서 / 이벤트 로그
sensors                              # lm-sensors 온도
smartctl -a /dev/sda                 # 디스크 SMART
dmidecode -t system                  # 장비 정보
```

2계층: 데이터 링크 계층 (Data Link)

MAC, ARP, 스위치 학습, VLAN, 본딩, 브리지, STP. 같은 서브넷 내 통신 문제를 다룹니다.

2-1 잘못된 ARP 엔트리

판단 IP·라우팅 정상인데 통신 불가, MAC 이 실제와 다름

```
C ip neigh replace $S_IP lladdr 00:11:22:33:44:55 dev $C_IF nud permanent
C ping -c 2 -W 1 $S_IP            # 실패
C ip neigh show; S ip link show $S_IF
C tcpdump -i $C_IF -n arp -c 5
C ip neigh del $S_IP dev $C_IF   # 복구
C ip neigh flush all                 # 전체 캐시 삭제
```

2-2 ARP 상태 해석

판단 REACHABLE/STALE/DELAY/PROBE 정상, INCOMPLETE/FAILED = 응답 없음(L1·L2 문제)

```
ip neigh show
ip neigh show nud failed
ip neigh show nud incomplete
arp -n
arping -I eth0 -c 3 <대상IP>          # L2 도달성 (ICMP 와 무관)
cat /proc/net/arp
```

2-3 IP/MAC 중복 충돌

판단 arping -D 에서 응답이 오면 IP 중복

```
arping -D -I eth0 -c 3 <IP>          # DAD: 중복 IP 검사
arping -I eth0 -c 3 <IP>             # 응답한 MAC 목록 확인(2개 이상이면 충돌)
ip neigh show | sort -k5 | awk '{print $5}' | uniq -d    # 같은 MAC 이 여러 IP
# MAC 중복 실험
S ip link set $S_IF down
S ip link set $S_IF address $(C cat /sys/class/net/$C_IF/address)
S ip link set $S_IF up
```

2-4 패킷 손실 · 프레임 드롭 · 지연 주입

판단 tc netem 으로 장애를 재현하고 통계로 확인

```
S tc qdisc add dev $S_IF root netem loss 30%
S tc qdisc change dev $S_IF root netem delay 100ms 20ms loss 5% duplicate 1% corrupt 0.1% reorder 10%
C ping -c 20 $S_IP | tail -3
S ip -s -s link show $S_IF          # errors/dropped/overruns/carrier/collisions
S tc -s qdisc show dev $S_IF
S tc qdisc del dev $S_IF root       # 복구
nstat -az | egrep -i "InErrors|InDiscards|OutDiscards"
cat /proc/net/dev
```

2-5 L2 트래픽 캡처 (ARP, 브로드캐스트, STP)

판단 ARP 폭주·루프 징후 = 브로드캐스트 스톰

```
tcpdump -i eth0 -n -e arp
tcpdump -i eth0 -n -e ether broadcast -c 100
tcpdump -i eth0 -n -e ether multicast
tcpdump -i eth0 -n -e -c 20 stp      # STP BPDU
tcpdump -i eth0 -n -e vlan           # 802.1Q 태그 확인
tcpdump -i eth0 -n -e ether host aa:bb:cc:dd:ee:ff
```

2-6 VLAN 설정 · 점검

판단 VLAN ID 불일치 = 같은 스위치인데 통신 불가

```
ip link add link eth0 name eth0.100 type vlan id 100
ip addr add 192.168.100.10/24 dev eth0.100; ip link set eth0.100 up
ip -d link show eth0.100             # VLAN 상세
cat /proc/net/vlan/config
tcpdump -i eth0 -n -e vlan 100
ip link del eth0.100
```

2-7 본딩(bonding)/팀 상태 점검

판단 slave 하나가 down 이면 이중화 깨짐, mode 불일치(LACP) 주의

```
cat /proc/net/bonding/bond0          # Mode, MII Status, Slave 상태, LACP Partner
ip -d link show bond0
nmcli con show; nmcli dev status
ip link set eth1 down; ip link set eth1 up      # slave failover 시험
dmesg | grep -i bond
```

2-8 브리지 · 스위칭 점검 (가상환경/KVM/Docker)

판단 MAC 학습 테이블 / 포트 상태

```
bridge link show
bridge fdb show
bridge vlan show
brctl show; brctl showmacs br0
bridge -s link show
```

2-9 MTU / 점보프레임 불일치 (L2~L3 걸침)

판단 작은 패킷 OK, 큰 패킷 실패 = MTU 불일치

```
S ip link set $S_IF mtu 1000
C ping -c 2 -s 100 $S_IP
C ping -c 2 -s 1400 -M do $S_IP   # 실패(DF 설정)
C tracepath $S_IP                 # pmtu
ip link show | grep mtu
S ip link set $S_IF mtu 1500        # 복구
ping -M do -s 8972 <IP>              # 점보프레임(9000) 경로 검증
```

3계층: 네트워크 계층 (Network)

IP 주소, 서브넷, 게이트웨이, 라우팅, ICMP, NAT, 방화벽(L3), IPv6.

3-1 IP/서브넷 설정 오류

판단 Network is unreachable → 경로 없음. 서브넷 마스크 확인

```
C ip addr flush dev $C_IF
C ip addr add 10.0.1.1/24 dev $C_IF
C ping -c 2 -W 1 $S_IP            # Network is unreachable
C ip addr; C ip route
C ip addr flush dev $C_IF; C ip addr add $C_IP/$PREFIX dev $C_IF   # 복구
```

3-2 게이트웨이/라우팅 누락

판단 ip route get 으로 실제 선택 경로를 확인

```
S ip addr add 192.168.100.1/24 dev lo
C ping -c 2 -W 1 192.168.100.1       # unreachable
C ip route get 192.168.100.1
C ip route add 192.168.100.0/24 via $S_IP
C ip route add default via $S_IP  # 기본 게이트웨이
C ip route del 192.168.100.0/24
ip route show table all
ip rule show                         # 정책 라우팅
ip route show table main
```

3-3 ICMP 만 차단 (ping 불가, 서비스 정상)

판단 ping 실패 ≠ 네트워크 장애

```
S iptables -A INPUT -p icmp -j DROP
C ping -c 2 -W 1 $S_IP
C curl -s -o /dev/null -w "%{http_code}\n" http://$S_IP:$PORT    # 200
S iptables -D INPUT -p icmp -j DROP
```

3-4 경로 추적 · 병목 구간 찾기

판단 * * * 는 ICMP 차단일 수 있음. mtr 로 구간별 손실 확인

```
traceroute -n 8.8.8.8
traceroute -T -p 443 -n <IP>         # TCP 로 추적(ICMP 차단 우회)
traceroute -I -n <IP>                # ICMP 모드
tracepath -n <IP>
mtr -rwzbc 100 8.8.8.8               # 리포트 모드, 손실·지연 통계
ping -c 100 -i 0.2 <IP> | tail -3
```

3-5 비대칭 라우팅 · rp_filter 로 인한 드롭

판단 여러 NIC 에서 응답이 다른 NIC 로 나가면 rp_filter 가 차단

```
sysctl net.ipv4.conf.all.rp_filter net.ipv4.conf.eth0.rp_filter
sysctl -w net.ipv4.conf.all.rp_filter=2     # loose 모드
nstat -az | grep -i IPReversePathFilter
sysctl -w net.ipv4.conf.all.log_martians=1; dmesg | grep -i martian
ip route get <외부IP> from <내IP>
```

3-6 IP 포워딩 · NAT (라우터/게이트웨이 역할)

판단 포워딩 꺼짐 또는 MASQUERADE 누락

```
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
```

```
iptables -t nat -L -n -v
iptables -t nat -A POSTROUTING -s $NET -o eth0 -j MASQUERADE
iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
conntrack -L | head; conntrack -S
```

3-7 IPv6 점검

판단 IPv6 우선 선택으로 지연이 생기는 경우도 많음

```
ip -6 addr; ip -6 route; ip -6 neigh
ping -6 -c 3 ::1
ping -6 -c 3 2001:4860:4860::8888
traceroute6 -n 2001:4860:4860::8888
curl -4 -sI https://example.com | head -1; curl -6 -sI https://example.com | head -1
sysctl net.ipv6.conf.all.disable_ipv6
rdisc6 eth0                          # Router Advertisement 확인
```

3-8 DHCP · 주소 할당 문제

판단 169.254.x.x 또는 주소 없음 = DHCP 응답 없음

```
dhclient -v eth0                     # 요청 과정 출력
dhclient -r eth0                     # 반납
tcpdump -i eth0 -n port 67 or port 68
journalctl -u NetworkManager | grep -i dhcp
nmcli device show eth0 | grep -i -E "ip4|gateway|dns"
cat /var/lib/dhcp/dhclient.leases
networkctl status eth0
```

3-9 ICMP 에러 메시지 해석

판단 원인 분류

```
# Destination Net Unreachable: 경로 없음 / Host Unreachable: ARP 실패
# Frag needed (type3 code4): MTU / TTL exceeded: 루프 또는 홉 초과
tcpdump -i eth0 -n icmp
tcpdump -i eth0 -n 'icmp[icmptype] != icmp-echo and icmp[icmptype] != icmp-echoreply'
ping -t 1 8.8.8.8                    # TTL=1 로 첫 홉 확인(ttl 지정)
```

4계층: 전송 계층 (Transport)

TCP/UDP 포트, 3-way handshake, 방화벽(L4), 소켓 상태, 포트/연결 고갈, 재전송.

4-1 포트 미리슨 (Connection refused)

판단 즉시 refused = 리스너 없음 또는 REJECT

```
C nc -zv -w 3 $S_IP 9999         # refused
S ss -tulnp | grep 9999
S python3 -m http.server 9999 --bind $S_IP &
C nc -zv -w 3 $S_IP 9999         # succeeded
ss -tlnp; ss -ulnp; ss -lntp sport = :80
lsof -i :80; fuser -v 80/tcp
```

4-2 방화벽 DROP vs REJECT

판단 타임아웃 = DROP/경로 문제, 즉시 거부 = REJECT/리스너 없음

```
S iptables -A INPUT -p tcp --dport $PORT -j DROP
time C nc -zv -w 5 $S_IP $PORT    # 5초 후 timed out
S iptables -D INPUT -p tcp --dport $PORT -j DROP
S iptables -A INPUT -p tcp --dport $PORT -j REJECT --reject-with tcp-reset
time C nc -zv -w 5 $S_IP $PORT    # 즉시 refused
S iptables -D INPUT -p tcp --dport $PORT -j REJECT --reject-with tcp-reset
```

4-3 방화벽 정책 확인 (iptables / nftables / firewalld / ufw)

판단 규칙 순서·기본 정책(policy) 확인

```
iptables -L -v -n --line-numbers; iptables -S
iptables -t nat -L -n -v
nft list ruleset
firewall-cmd --state; firewall-cmd --list-all
firewall-cmd --permanent --add-port=$PORT/tcp && firewall-cmd --reload
ufw status verbose
iptables -I INPUT 1 -p tcp --dport $PORT -j ACCEPT   # 임시 허용
iptables -A INPUT -j LOG --log-prefix "DROP: "      # 드롭 로깅
journalctl -k | grep "DROP: "
```

4-4 3-Way Handshake 관찰

판단 SYN 만 보이고 SYN-ACK 없으면 서버/방화벽 문제, SYN-ACK 는 있는데 ACK 없으면 반대 방향 문제

```
S tcpdump -i $S_IF -n "tcp port $PORT" -c 10
C curl -s -o /dev/null http://$S_IP:$PORT
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-syn != 0'
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-rst != 0'      # RST 추적
tcpdump -i eth0 -nn 'tcp[tcpflags] == tcp-syn'          # SYN 만
tcpdump -i eth0 -nn 'tcp[tcpflags] & (tcp-syn|tcp-ack) == (tcp-syn|tcp-ack)'  # SYN-ACK
```

4-5 소켓 상태 분석

판단 TIME_WAIT 다수=정상 범위 점검, CLOSE_WAIT 다수=앱이 close 안 함, SYN_RECV 다수=SYN flood 의심

```
ss -s
ss -tan | awk 'NR>1{print $1}' | sort | uniq -c | sort -rn
ss -tan state time-wait | wc -l
ss -tan state close-wait | wc -l
ss -tan state syn-recv | wc -l
ss -tanp state close-wait             # 어느 프로세스인가
ss -tan | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
ss -tnoi dst $S_IP                # rtt, cwnd, 재전송, 타이머
netstat -ant | awk '{print $6}' | sort | uniq -c
```

4-6 재전송 · 지연 · 처리량

판단 retrans 증가 = 손실/혼잡. 대역폭과 지연을 직접 측정

```
nstat -az | egrep "TcpRetransSegs|TcpExtTCPLostRetransmit|TcpExtTCPTimeouts|TcpExtTCPSynRetrans"
netstat -s | egrep -i "retrans|timeout|failed|overflow|drop"
ss -ti | grep -E "retrans|rtt"
S iperf3 -s &
C iperf3 -c $S_IP -t 10          # TCP 처리량
C iperf3 -c $S_IP -u -b 100M     # UDP 손실/지터
C iperf3 -c $S_IP -P 4 -R        # 병렬·역방향
```

4-7 포트·소켓 고갈 / conntrack 가득 참

판단 Cannot assign requested address, nf_conntrack: table full

```
sysctl net.ipv4.ip_local_port_range
ss -tan | wc -l
sysctl net.ipv4.tcp_tw_reuse net.ipv4.tcp_fin_timeout
cat /proc/sys/net/netfilter/nf_conntrack_{count,max}
dmesg | grep -i "nf_conntrack: table full"
sysctl -w net.netfilter.nf_conntrack_max=262144
ulimit -n; cat /proc/sys/fs/file-nr; cat /proc/sys/fs/file-max
ls /proc/<PID>/fd | wc -l            # 프로세스 FD 사용량
```

4-8 백로그 · SYN flood 점검

판단 리슨 큐 넘침 → 접속 지연/실패

```
ss -ltn                              # Recv-Q(현재 대기) / Send-Q(백로그 한도)
nstat -az | egrep "ListenOverflows|ListenDrops|TCPReqQFullDrop|TCPReqQFullDoCookies"
sysctl net.core.somaxconn net.ipv4.tcp_max_syn_backlog net.ipv4.tcp_syncookies
sysctl -w net.ipv4.tcp_syncookies=1
dmesg | grep -i "SYN flooding"
```

4-9 UDP 포트 점검

판단 UDP 는 응답이 없어 판단이 어려움 → ICMP port unreachable 로 판단

```
nc -zvu <IP> 53
ss -ulnp
nmap -sU -p 53,123,161 <IP>
tcpdump -i eth0 -n udp port 53
S nc -ul 5000 &
echo hi | C nc -u -w1 $S_IP 5000
```

4-10 포트 스캔 · 서비스 노출 확인

판단 내 서버가 외부에 무엇을 열고 있는지 (허가된 대상에만 사용)

```
nmap -sT -p- <IP>                    # 전체 TCP
nmap -sV -p 22,80,443 <IP>           # 서비스/버전
nmap -Pn -p 443 <IP>                 # ping 차단 환경
ss -tulnp                            # 로컬에서 직접 확인
nc -zv <IP> 20-25
```

5계층: 세션 계층 (Session)

세션 수립·유지·종료, 타임아웃, keepalive, NFS/RPC, 로그인 세션.

5-1 수립된 세션과 타이머

판단 timer:(keepalive,...) / (on,...) 로 재전송·유휴 확인

```
S python3 -m http.server $PORT --bind $S_IP &
C nc $S_IP $PORT &
S ss -o state established "( sport = :$PORT )"
S ss -tnoi state established
ss -tn state established | wc -l
```

5-2 세션 중간 단절 · 재전송 관찰

판단 응답 경로 DROP 시 재전송 타이머가 증가

```
S iptables -A OUTPUT -p tcp --sport $PORT -j DROP
echo "GET / HTTP/1.0" | C nc -w 2 $S_IP $PORT
C ss -tno state established          # timer:(on,...,retrans) 확인
S iptables -D OUTPUT -p tcp --sport $PORT -j DROP
cat /proc/sys/net/ipv4/tcp_retries2
```

5-3 Keepalive · 유휴 세션 끊김 대응

판단 방화벽/NAT/LB 의 idle timeout 이 keepalive 주기보다 짧으면 세션 끊김

```
sysctl net.ipv4.tcp_keepalive_time net.ipv4.tcp_keepalive_intvl net.ipv4.tcp_keepalive_probes
sysctl -w net.ipv4.tcp_keepalive_time=300
ssh -o ServerAliveInterval=30 -o ServerAliveCountMax=3 user@host
grep -E "ClientAlive|TCPKeepAlive" /etc/ssh/sshd_config
ss -tnoe state established | grep keepalive
```

5-4 NFS / RPC 세션

판단 hard/soft 마운트, stale file handle, portmapper 문제

```
rpcinfo -p <NFS서버>
showmount -e <NFS서버>
mount | grep nfs; nfsstat -m; nfsstat -c
mount -t nfs -o vers=4,timeo=600,retrans=2 <서버>:/export /mnt
systemctl status nfs-server rpcbind
umount -f -l /mnt                   # 응답 없는 마운트 강제 해제
dmesg | grep -i -E "nfs|rpc"
exportfs -v
```

5-5 로그인 세션 관리

판단 누가 접속 중인지, 세션 정리

```
who -a; w; last -n 20; lastb | head
loginctl list-sessions; loginctl show-session <ID>
loginctl terminate-session <ID>
pkill -KILL -t pts/2
ss -tnp state established '( sport = :22 )'
tmux ls; screen -ls                 # 끊김 대비 세션 유지 도구
```

5-6 SMB / LDAP / DB 세션

판단 세션 기반 서비스의 연결 상태

```
smbstatus; smbclient -L //<서버> -N
ldapsearch -x -H ldap://<서버> -b "" -s base
```

```
mysql -e "SHOW PROCESSLIST; SHOW STATUS LIKE 'Threads_connected'"
psql -c "select state,count(*) from pg_stat_activity group by 1;"
ss -tn dport = :3306 | wc -l
```

6계층: 표현 계층 (Presentation)

TLS/SSL 인증서, 암호군, 프로토콜 버전, 문자 인코딩, 압축, 직렬화 형식.

6-1 TLS 서버 구성 및 핸드셰이크 확인

판단 인증서 체인·SAN·만료일 확인

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout $KEY -out $CRT -days 30 -subj "/CN=$TLS_CN"
S openssl s_server -accept $TLS_PORT -cert $CRT -key $KEY -www &
C openssl s_client -connect $S_IP:$TLS_PORT -servername $TLS_CN </dev/null 2>&1 | head -30
C openssl s_client -connect $S_IP:$TLS_PORT </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer
-dates -ext subjectAltName
C curl -vk https://$S_IP:$TLS_PORT/ 2>&1 | egrep "SSL|TLS|subject|expire|issuer"
```

6-2 인증서 만료 점검

판단 -checkend 는 종료 코드로 판정 (모니터링에 활용)

```
openssl x509 -in $CRT -noout -enddate
openssl x509 -in $CRT -noout -checkend 86400 && echo "24h 이상 유효" || echo "24h 내 만료"
openssl x509 -in $CRT -noout -checkend 0 && echo "유효" || echo "만료됨"
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noou
t -enddate
for h in a.com b.com; do echo -n "$h "; echo | openssl s_client -connect $h:443 -servername $h 2>/dev/nul
l | openssl x509 -noout -enddate; done
```

6-3 프로토콜/암호군 불일치

판단 handshake failure / no protocols available

```
C openssl s_client -connect $S_IP:$TLS_PORT -tls1_2 </dev/null 2>&1 | grep -E "Protocol|Cipher"
C openssl s_client -connect $S_IP:$TLS_PORT -tls1_3 </dev/null 2>&1 | grep -E "Protocol|Cipher"
C openssl s_client -connect $S_IP:$TLS_PORT -tls1_1 </dev/null 2>&1 | tail -5
openssl ciphers -v 'HIGH:!aNULL' | head
nmap --script ssl-enum-ciphers -p 443 <IP>
curl --tlsv1.2 --tls-max 1.2 -vI https://<호스트>
```

6-4 신뢰 체인 · 호스트명 불일치 · 시스템 CA

판단 (60) SSL certificate problem

```
C curl -v https://$S_IP:$TLS_PORT/ 2>&1 | grep -iE "certificate|verify|SSL"
C curl --cacert $CRT --resolve $TLS_CN:$TLS_PORT:$S_IP https://$TLS_CN:$TLS_PORT/ -o /dev/null -w "%{http
_code}\n"
openssl verify -CAfile ca.pem server.pem
openssl s_client -connect <호스트>:443 -showcerts </dev/null    # 중간 인증서 누락 확인
update-ca-certificates               # Debian/Ubuntu
update-ca-trust extract              # RHEL 계열
curl -v --cacert /etc/ssl/certs/ca-certificates.crt https://<호스트>
date; timedatectl                    # 시간 오차도 인증서 오류의 흔한 원인
```

6-5 SSH 호스트키 · 키 형식 문제

판단 REMOTE HOST IDENTIFICATION HAS CHANGED

```
ssh-keyscan -t ed25519,rsa <호스트>
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub     # 핑거프린트
ssh-keygen -R <호스트>               # known_hosts 에서 제거
ssh -vvv -o PreferredAuthentications=publickey user@host
ssh-keygen -y -f ~/.ssh/id_ed25519   # 개인키로 공개키 복원
ssh -Q cipher; ssh -Q kex            # 지원 알고리즘
```

6-6 문자셋/인코딩 깨짐

판단 한글이 ???, 깨짐 → locale/파일 인코딩 확인

```
locale; locale -a | egrep -i "utf|euc"
echo "안녕하세요" > /tmp/utf8.txt
iconv -f UTF-8 -t EUC-KR /tmp/utf8.txt > /tmp/euckr.txt
file -i /tmp/utf8.txt /tmp/euckr.txt
cat /tmp/euckr.txt                  # 깨짐
iconv -f EUC-KR -t UTF-8 /tmp/euckr.txt
localectl status; localectl set-locale LANG=ko_KR.UTF-8
export LANG=en_US.UTF-8 LC_ALL=en_US.UTF-8
convmv -f euc-kr -t utf-8 -r --notest .        # 파일명 인코딩 변환
curl -sI <URL> | grep -i content-type          # charset 확인
```

6-7 압축 · 직렬화 · 데이터 형식 검증

판단 Content-Encoding, JSON/XML 형식 오류

```
curl -sI -H "Accept-Encoding: gzip" <URL> | grep -i content-encoding
curl -s --compressed <URL> | head
echo '{"a":1' | python3 -m json.tool         # 문법 오류 확인
jq . file.json; xmllint --noout file.xml
gzip -t file.gz; tar -tzf file.tgz >/dev/null
base64 -d <<< "aGVsbG8="; file unknown.bin
```

7계층: 응용 계층 (Application)

HTTP, DNS, SSH, 메일, DB, 웹서버/프록시/LB, 서비스 프로세스와 로그.

7-1 HTTP 상태 코드 · 응답시간 분해

판단 느림의 원인을 DNS/연결/TLS/서버처리/전송으로 분리

```
C curl -sI http://$S_IP:$PORT/ | head -1        # 200
C curl -sI http://$S_IP:$PORT/nope | head -1     # 404
C curl -o /dev/null -s -w "dns:%{time_namelookup} tcp:%{time_connect} tls:%{time_appconnect} ttfb:%{time_
starttransfer} total:%{time_total} code:%{http_code}\n" http://$S_IP:$PORT/
curl -v https://<호스트>/ 2>&1 | head -40
curl -L -I https://<호스트>/                      # 리다이렉트 추적
curl -H "Host: a.example.com" http://<IP>/       # 가상호스트 테스트
curl -X POST -d '{"a":1}' -H 'Content-Type: application/json' <URL>
curl --max-time 5 --retry 3 <URL>
chmod 000 $WEBROOT/index.html; C curl -sI http://$S_IP:$PORT/ | head -1; chmod 644 $WEBROOT/index.html
```

7-2 상태 코드별 대응 가이드

판단 코드에 따른 1차 조치

```
# 301/302: 리다이렉트 루프 → curl -L -v 로 Location 확인
# 400: 요청 형식/헤더 크기 / 401·403: 인증·권한·ACL·SELinux
# 404: 경로·DocumentRoot / 408·504: 타임아웃(프록시↔업스트림)
# 413: 업로드 크기 제한(client_max_body_size) / 429: rate limit
# 500: 앱 오류 → 앱 로그 / 502: 업스트림 다운·응답 이상 / 503: 과부하·점검
getenforce; ausearch -m avc -ts recent             # SELinux 거부
ls -ldZ /var/www/html; restorecon -Rv /var/www/html
namei -l /var/www/html/index.html                  # 경로상 권한 확인
nginx -T | grep -E "client_max_body_size|proxy_read_timeout|upstream"
```

7-3 DNS 장애 진단

판단 ping 8.8.8.8 OK + ping 도메인 FAIL = DNS 문제

```
cat /etc/resolv.conf; resolvectl status | head -20
getent hosts example.com             # 시스템 리졸버 경로 그대로(nsswitch 포함)
dig example.com; dig +short example.com
dig @8.8.8.8 example.com             # 서버 비교
dig +trace example.com               # 루트부터 위임 추적
dig example.com MX / NS / TXT / AAAA / CNAME / SOA
dig -x 8.8.8.8                       # 역방향
dig +tcp example.com; dig +dnssec example.com
dig @10.255.255.1 example.com +time=2 +tries=1   # 타임아웃 재현
nslookup example.com 8.8.8.8
host -a example.com
grep hosts /etc/nsswitch.conf; cat /etc/hosts
resolvectl query example.com; resolvectl flush-caches
systemd-resolve --statistics
tcpdump -i eth0 -n port 53
time getent hosts example.com        # 해석 지연 측정
```

7-4 서비스 프로세스 (systemd)

판단 서비스 다운이면 Connection refused 로 4계층처럼 보임 → ss 로 리스너 확인

```
systemctl status nginx; systemctl is-active nginx; systemctl is-enabled nginx
systemctl --failed
systemctl restart nginx; systemctl reload nginx
journalctl -u nginx -n 50 --no-pager
journalctl -u nginx --since "10 min ago" -p err
journalctl -xe
nginx -t; apachectl configtest; sshd -t; named-checkconf; haproxy -c -f /etc/haproxy/haproxy.cfg
```

```
systemctl cat nginx; systemctl list-dependencies nginx
systemctl stop nginx; curl -sI http://localhost | head -1; ss -tlnp | grep ':80 '; systemctl start nginx
ps aux --sort=-%cpu | head; ps -eo pid,ppid,stat,cmd | grep -w Z   # 좀비
dmesg | grep -i -E "oom|killed process"
```

7-5 로그 분석

판단 상태코드 분포, 상위 IP, 에러 패턴, 시간대별 급증

```
tail -f /var/log/nginx/error.log
awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
grep -c ' 5[0-9][0-9] ' /var/log/nginx/access.log
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head
awk '{print substr($4,2,14)}' /var/log/nginx/access.log | uniq -c | sort -rn | head    # 분 단위 요청량
awk '$NF > 2 {print $7,$NF}' access.log | sort -k2 -nr | head                          # 느린 요청(응답시간
필드)
grep -iE "error|fail|denied|timeout|refused" /var/log/syslog | tail -20
journalctl -p err -b --no-pager | tail -30
journalctl --since "2026-11-11 00:00" --until "2026-11-11 01:00"
zgrep "ERROR" /var/log/app/*.gz
lnav /var/log/syslog                # 설치되어 있는 경우
```

7-6 SSH 접속 장애

판단 단계: 도달성 → 포트 → 핸드셰이크 → 인증

```
ssh -vvv user@host
nc -zv host 22; ss -tlnp | grep sshd
sshd -T | egrep "^(port|permitrootlogin|passwordauthentication|pubkeyauthentication|allowusers|allowgroup
s|maxauthtries)"
journalctl -u sshd -n 30 --no-pager; tail /var/log/auth.log /var/log/secure
lastb | head; last | head
ls -ld ~ ~/.ssh ~/.ssh/authorized_keys     # 권한: 700 / 600
chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys
restorecon -Rv ~/.ssh                     # SELinux
ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 user@host true
fail2ban-client status sshd; fail2ban-client set sshd unbanip <IP>
ssh -J jump user@target; ssh -L 8080:localhost:80 user@host
```

7-7 웹서버 · 리버스 프록시 · LB 점검

판단 502/504 는 프록시와 업스트림 사이 문제

```
nginx -T | head -100; nginx -s reload
curl -sI http://127.0.0.1:<업스트림포트>/           # 업스트림 직접 호출
tail -f /var/log/nginx/error.log | grep -i upstream
apachectl -S; apachectl -t -D DUMP_MODULES
echo "show stat" | socat stdio /var/run/haproxy.sock | cut -d, -f1,2,18 | column -s, -t
ab -n 1000 -c 50 http://<호스트>/                  # 부하 테스트
wrk -t4 -c100 -d30s http://<호스트>/
curl -w "@curl-format.txt" -o /dev/null -s <URL>
```

7-8 DB · 메일 · 기타 서비스

판단 포트 → 인증 → 쿼리 순서로

```
mysqladmin -h <호스트> -u root -p ping; mysql -h <호스트> -e "select 1"
pg_isready -h <호스트> -p 5432
redis-cli -h <호스트> ping; redis-cli info clients
nc -zv <호스트> 25 587 993
openssl s_client -starttls smtp -connect <호스트>:587
swaks --to a@b.com --server <호스트>
postqueue -p; tail /var/log/mail.log
mongosh --host <호스트> --eval 'db.runCommand({ping:1})'
curl -s localhost:9200/_cluster/health?pretty      # Elasticsearch
ntpq -p; chronyc tracking; timedatectl             # 시간 동기화
```

7-9 시간 동기화 · 로그 시각 불일치 (자주 간과)

판단 인증서·Kerberos·로그 상관분석에 영향

```
timedatectl status
chronyc sources -v; chronyc tracking
ntpq -pn
hwclock --show
timedatectl set-ntp true
```

부록 A. 시스템 자원 병목 (네트워크처럼 보이는 장애)

CPU, 메모리, 디스크, 파일 디스크립터 문제가 네트워크 지연·타임아웃으로 나타나는 경우가 많습니다.

A-1 CPU · 부하 · 인터럽트

판단 softirq 가 한 코어에 몰리면 패킷 드롭

```
uptime; top -bn1 | head -15; mpstat -P ALL 1 3; vmstat 1 5
pidstat 1 3
cat /proc/softirqs | egrep "NET_RX|NET_TX"
cat /proc/interrupts | grep eth0
cat /proc/net/softnet_stat           # 두 번째 열(dropped) 증가 여부
ethtool -l eth0; ethtool -g eth0     # 큐 수 / 링버퍼 크기
ethtool -G eth0 rx 4096              # 링버퍼 확대
sysctl net.core.netdev_max_backlog net.core.rmem_max net.core.wmem_max
```

A-2 메모리 · OOM

판단 OOM 으로 서비스가 죽으면 갑자기 refused

```
free -h; vmstat 1 5; cat /proc/meminfo | head -20
ps aux --sort=-%mem | head
dmesg -T | grep -i -E "out of memory|oom-killer|killed process"
journalctl -k | grep -i oom
slabtop -o | head; sysctl vm.swappiness vm.overcommit_memory
smem -tk 2>/dev/null | tail
```

A-3 디스크 · I/O · inode

판단 디스크 가득 → 로그/세션/소켓 파일 생성 실패

```
df -h; df -i
du -xh / --max-depth=2 2>/dev/null | sort -rh | head
lsof +L1                             # 삭제됐지만 열려 있는 파일
iostat -xz 1 3; iotop -oPa
lsblk; mount | grep -E "ro,|errors"
dmesg | grep -i -E "i/o error|ext4|xfs|read-only"
```

A-4 프로세스 · 파일 디스크립터 · 스레드

판단 Too many open files

```
ulimit -a; cat /proc/<PID>/limits
ls /proc/<PID>/fd | wc -l
lsof -p <PID> | wc -l; lsof -i -nP | awk '{print $1}' | sort | uniq -c | sort -rn
cat /proc/sys/fs/file-nr
strace -f -p <PID> -e trace=network -tt   # 시스템콜 추적
perf top; ltrace -p <PID>
systemctl show nginx -p LimitNOFILE
```

A-5 커널 네트워크 파라미터 일괄 점검

판단 튜닝 전 현재값 기록

```
sysctl -a 2>/dev/null | egrep "net\.(core|ipv4\.tcp|ipv4\.ip_local|netfilter)" | head -80
sysctl net.ipv4.tcp_congestion_control net.core.default_qdisc
sysctl net.ipv4.tcp_rmem net.ipv4.tcp_wmem
sysctl -p /etc/sysctl.conf; sysctl --system
nstat -az | head -50; cat /proc/net/snmp; cat /proc/net/netstat
```

부록 B. 패킷 캡처 · 분석 종합

어느 계층 문제인지 불확실할 때 가장 확실한 방법은 패킷을 직접 보는 것입니다.

B-1 tcpdump 필터 모음

판단 캡처 후 Wireshark 로 분석

```
tcpdump -i any -nn host $S_IP
tcpdump -i eth0 -nn port 443 and host <IP>
tcpdump -i eth0 -nn net $NET
tcpdump -i eth0 -nn 'tcp port 80 and (tcp[tcpflags] & tcp-syn != 0)'
tcpdump -i eth0 -nn -A 'tcp port 80' -c 20          # ASCII 출력(평문 HTTP)
tcpdump -i eth0 -nn -XX -c 5                        # 헥사 덤프
tcpdump -i eth0 -nn not port 22                     # SSH 제외
tcpdump -i eth0 -nn 'icmp or arp'
tcpdump -i eth0 -nn -s0 -w /tmp/cap.pcap -C 100 -W 5   # 100MB x 5개 순환
tcpdump -r /tmp/cap.pcap -nn | head
tcpdump -i eth0 -nn 'ip[8] < 5'                     # TTL 낮은 패킷
tcpdump -i eth0 -nn 'greater 1400'                  # 큰 패킷
```

B-2 tshark / 기타 분석 도구

판단 통계·대화 단위 분석

```
tshark -r /tmp/cap.pcap -q -z conv,tcp
tshark -r /tmp/cap.pcap -Y "tcp.analysis.retransmission" | head
tshark -r /tmp/cap.pcap -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri
tshark -r /tmp/cap.pcap -Y "dns" -T fields -e dns.qry.name -e dns.flags.rcode
tshark -r /tmp/cap.pcap -q -z io,stat,1
tshark -r /tmp/cap.pcap -Y "tls.handshake.type==1" -T fields -e tls.handshake.extensions_server_name
ngrep -d eth0 -q "GET" port 80
iftop -i eth0; nload eth0; iptraf-ng
nethogs eth0                          # 프로세스별 대역폭
bmon; vnstat -l
```

B-3 실시간 트래픽 · 대역폭

판단 누가 대역을 쓰는가

```
sar -n DEV 1 5; sar -n TCP,ETCP 1 5; sar -n SOCK 1 3
ip -s link show eth0
watch -n1 "cat /proc/net/dev"
ss -tin | grep -E "bytes_sent|bytes_acked"
cat /sys/class/net/eth0/statistics/{rx_bytes,tx_bytes,rx_dropped,tx_dropped,rx_errors,tx_errors}
dstat -n 1 5
```

부록 C. 컨테이너 · 가상화 · 클라우드 환경

Docker/Kubernetes/클라우드에서는 L2~L4 가 오버레이·iptables·보안그룹으로 추상화됩니다.

C-1 Docker

판단 컨테이너 네트워크 확인

```
docker ps; docker network ls; docker network inspect bridge
docker inspect -f '{{.NetworkSettings.IPAddress}}' <컨테이너>
docker exec -it <컨테이너> sh -c "ip a; ip r; cat /etc/resolv.conf; ss -tlnp"
docker logs --tail 100 -f <컨테이너>
docker port <컨테이너>
nsenter -t $(docker inspect -f '{{.State.Pid}}' <컨테이너>) -n ss -tlnp   # 호스트 도구로 컨테이너 네트워크 확인
iptables -t nat -L DOCKER -n -v
docker stats --no-stream; docker events --since 10m
```

C-2 Kubernetes

판단 Service → Endpoint → Pod → DNS 순으로

```
kubectl get pods -A -o wide; kubectl describe pod <파드>
kubectl logs <파드> --previous; kubectl get events --sort-by=.lastTimestamp
kubectl get svc,ep,ingress -A
kubectl exec -it <파드> -- sh -c "nslookup kubernetes.default; curl -sI http://<svc>:<port>"
kubectl run tmp --rm -it --image=nicolaka/netshoot -- bash     # 디버그 파드
kubectl get networkpolicy -A
kubectl -n kube-system logs -l k8s-app=kube-dns
kubectl top nodes; kubectl top pods
kubectl port-forward svc/<svc> 8080:80
kubectl debug node/<노드> -it --image=busybox
```

C-3 클라우드 (AWS 예시)

판단 OS 내부 점검 + 보안그룹/NACL/라우팅 테이블

```
curl -s http://169.254.169.254/latest/meta-data/local-ipv4      # IMDS(v1 허용 시)
aws ec2 describe-security-groups --group-ids <sg-id>
aws ec2 describe-route-tables --filters Name=vpc-id,Values=<vpc-id>
aws ec2 describe-network-acls
aws ec2 describe-instance-status --instance-ids <id>
aws ec2 get-console-output --instance-id <id>                   # 부팅 로그
# 4계층 도달 확인: OS 방화벽 + 보안그룹 + NACL 세 곳을 모두 점검
VPC Reachability Analyzer 사용(콘솔/CLI)
```

C-4 VPN · 터널 · 오버레이

판단 MTU 와 시계열 핸드셰이크 확인

```
wg show; ip -d link show wg0
ipsec statusall; ip xfrm state; ip xfrm policy
ip tunnel show; ip -d link show type vxlan
ping -M do -s 1372 <터널건너편IP>      # 터널 MTU 확인
openvpn --config client.ovpn --verb 4
```

부록 D. 보안 관점 점검 (침입·공격 징후)

DDoS, 무차별 대입, 의심 연결, 비인가 포트 등. 자신이 관리하는 시스템에만 사용하세요.

D-1 의심 연결 · 비인가 포트

판단 알 수 없는 리슨 포트/외부 연결 확인

```
ss -tulnp; ss -tnp state established
lsof -i -nP | grep -v 127.0.0.1
netstat -tunap | grep -v LISTEN | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
ps auxf; ls -la /tmp /var/tmp /dev/shm
crontab -l; ls /etc/cron.*; systemctl list-timers
find / -xdev -mtime -1 -type f 2>/dev/null | head
rkhunter --check; chkrootkit
```

D-2 무차별 대입 · 로그인 시도 분석

판단 공격 IP 추출 및 차단

```
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head
lastb | awk '{print $3}' | sort | uniq -c | sort -rn | head
journalctl -u sshd | grep -i "invalid user" | tail
iptables -I INPUT -s <공격IP> -j DROP
nft add rule inet filter input ip saddr <공격IP> drop
fail2ban-client status; fail2ban-client status sshd
ipset create block hash:ip; ipset add block <IP>; iptables -I INPUT -m set --match-set block src -j DROP
```

D-3 DDoS / 트래픽 폭주 대응

판단 SYN flood, 연결 수 제한, 레이트 리밋

```
ss -tan state syn-recv | wc -l
ss -tan | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
sysctl -w net.ipv4.tcp_syncookies=1
iptables -A INPUT -p tcp --syn -m limit --limit 50/s --limit-burst 100 -j ACCEPT
iptables -A INPUT -p tcp --syn -j DROP
iptables -A INPUT -p tcp --dport 80 -m connlimit --connlimit-above 50 -j REJECT
iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 5/s -j ACCEPT
tcpdump -i eth0 -nn -c 1000 | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -rn | head
```

D-4 감사 · 무결성

판단 파일 변경과 접근 기록

```
auditctl -l; ausearch -ts today -m USER_LOGIN
aide --check; rpm -Va | head; debsums -c
last -F | head; lastlog | head
getent passwd | awk -F: '$3==0'      # UID 0 계정
awk -F: '($2=="") {print $1}' /etc/shadow   # 빈 비밀번호
```

부록 E. 종합 진단 자동화 스크립트

아래에서 위로(L1→L7) 점검하는 한 방 스크립트와, 로그를 수집해 보고서로 남기는 스크립트입니다.

E-1 diag.sh: 계층별 자동 점검

판단 사용: ./diag.sh <대상IP> <포트> [도메인]

```
#!/bin/bash
IF=${IF:-eth0}; GW=$(ip route | awk '/default/{print $3; exit}')
T=$1; P=$2; D=$3
echo "[L1] 링크:";   ip -br link show $IF; ethtool $IF 2>/dev/null | egrep "Link detected|Speed|Duplex"
echo "[L2] ARP:";    ip neigh show | head -5; ip -s link show $IF | grep -A1 -E "RX:|TX:"
echo "[L3] IP/GW:";  ip -br addr show $IF; ip route show default
echo -n "[L3] GW ping: ";  ping -c 2 -W 1 $GW >/dev/null && echo OK || echo FAIL
echo -n "[L3] 대상 ping: "; ping -c 2 -W 1 $T >/dev/null && echo OK || echo "FAIL (ICMP 차단 가능성)"
echo -n "[L4] 포트 $P: ";   nc -zv -w 3 $T $P 2>&1 | tail -1
[ -n "$D" ] && { echo "[L7] DNS:"; dig +short $D | head -3; }
[ "$P" = "443" ] && { echo "[L6] TLS:"; echo | openssl s_client -connect $T:443 ${D:+-servername $D} 2>/d
ev/null | openssl x509 -noout -dates; }
echo "[L7] HTTP:";   curl -sI -m 5 ${D:+--resolve $D:$P:$T} "http://${D:-$T}:$P/" | head -1
```

E-2 장애 증거 일괄 수집

판단 장애 발생 직후 실행해 두면 사후 분석에 유용

```
#!/bin/bash
OUT=/tmp/incident-$(hostname)-$(date +%Y%m%d-%H%M%S); mkdir -p $OUT
{ date; uptime; ip -br addr; ip route; ip neigh; ip -s link; ss -s; ss -tanp; } > $OUT/net.txt 2>&1
{ free -h; df -h; df -i; vmstat 1 3; top -bn1 | head -30; } > $OUT/res.txt 2>&1
{ nstat -az; netstat -s; cat /proc/net/softnet_stat; } > $OUT/stats.txt 2>&1
dmesg -T | tail -300 > $OUT/dmesg.txt
journalctl -p warning --since "1 hour ago" --no-pager > $OUT/journal.txt 2>&1
iptables-save > $OUT/iptables.txt 2>&1; nft list ruleset > $OUT/nft.txt 2>&1
timeout 30 tcpdump -i any -nn -c 2000 -w $OUT/cap.pcap 2>/dev/null
tar czf $OUT.tgz -C /tmp $(basename $OUT); echo "수집 완료: $OUT.tgz"
```

E-3 지속 모니터링 한 줄 명령

판단 간헐적 장애 포착용

```
while true; do echo "$(date +%T) $(ping -c1 -W1 8.8.8.8 >/dev/null && echo up || echo DOWN)"; sleep 1; do
ne | tee ping.log
watch -n1 'ss -s; echo; nstat -az | egrep "Retrans|Drop"'
while sleep 5; do curl -o /dev/null -s -w "%{time_total} %{http_code}\n" http://<호스트>/; done
journalctl -f -p warning
ping -D -i 0.5 <IP> | awk '{print strftime("%T"), $0}' >> ping.log
```

부록 F. 증상 → 계층 빠른 판별표

증상	의심 계층	확인할 것
ethtool: Link detected: no	L1	케이블/포트/NIC, ethtool, dmesg
같은 대역인데 ping 불가, neigh FAILED/INCOMPLETE	L2	ip neigh, arping, VLAN, 스위치
ip neigh 에 중복 MAC / arping -D 응답	L2	IP·MAC 충돌, 브리지 루프
Network is unreachable	L3	ip addr, ip route, 서브넷·게이트웨이
ping 8.8.8.8 OK, ping 도메인 FAIL	L7	DNS: dig, resolv.conf, nsswitch
작은 패킷 OK, 큰 전송 중 멈춤	L2/L3	MTU: ping -M do -s, tracepath
즉시 Connection refused	L4/L7	ss -tlnp, systemctl status (서비스 다운)
30초 뒤 timed out	L3/L4	iptables DROP, 보안그룹, 경로, tcpdump
SYN 만 가고 SYN-ACK 없음	L3/L4	방화벽, 서버 다운, 경로
CLOSE_WAIT 수천 개	L4/L7	앱이 close() 미호출, 앱 로그/덤프
SYN_RECV 급증	L4	SYN flood, backlog, syncookies
nf_conntrack: table full	L3/L4	conntrack_max 증설, 타임아웃 조정
Cannot assign requested address	L4	포트 고갈, ip_local_port_range, tw_reuse
유휴 후 SSH/DB 연결 끊김	L5	keepalive, 방화벽/LB idle timeout
NFS stale file handle / 응답 없음	L5	rpcinfo, nfsstat, 마운트 옵션
SSL certificate problem (60)	L6	openssl s_client, 만료/체인/호스트명/시간
handshake failure	L6	프로토콜/암호군 불일치, SNI
한글이 ??? 로 표시	L6	locale, file -i, iconv
HTTP 502 Bad Gateway	L7	업스트림 다운, 프록시 로그
HTTP 504 Gateway Timeout	L7	업스트림 지연, proxy_read_timeout
HTTP 403	L7	권한/ACL/SELinux, namei -l
서비스 간헐적 느림	전계층	curl -w 분해, mtr, softnet_stat, 자원 병목
패킷 드롭 증가 (softnet dropped)	L1~L3	링버퍼, IRQ 분산, netdev_max_backlog

부록 G. 계층별 핵심 명령어 한 장 요약

계층	핵심 명령어
L1	`ip -br link | ethtool eth0 | ethtool -S eth0 | dmesg -T | grep -i link`
L2	`ip neigh | arping -I eth0 <IP> | ip -s link | tcpdump -e arp | bridge fdb`
L3	`ip addr | ip route get <IP> | ping | traceroute -n | mtr -rwzbc 100 | ping -M do -s`
L4	`ss -tulnp | ss -s | nc -zv <IP> <PORT> | iptables -L -n -v | nft list ruleset | nstat -az`
L5	`ss -o state established | ss -tnoi | sysctl net.ipv4.tcp_keepalive_* | nfsstat | who -a`
L6	`openssl s_client -connect H:443 | openssl x509 -checkend | locale | iconv | file -i`
L7	`curl -v / -w | dig +trace | systemctl status | journalctl -u | tail -f /var/log/...`
전체	`tcpdump -w cap.pcap | tshark -z conv,tcp | sar -n DEV,TCP | vmstat | iostat | dmesg`

진단 원칙

1. 증상 재현 및 범위 파악 (누가, 어디서, 언제부터, 무엇이 안 되는가)
2. L1→L7 순서로 정상/비정상 경계를 찾는다
3. 가설을 세우고 한 번에 하나만 변경한다
4. 변경 전 설정을 백업하고, 복구 방법을 먼저 정한다
5. 원인과 조치를 기록해 재발을 방지한다

부록 H. 일방향 벡터·패턴·바이트 다리
변수분리 대응 총정리

변수분리(분할, 전역 상수, 매개변수, 불리언 수축, 역방향 탐색)를 OSI 7계층과 리눅스 명령에 대응시킨 내용을 한데 모은 부록입니다. 철학의 근거는 **검증기_철학_취합2.pdf**(새 폴더.zip)에서 찾았고, 모든 코드와 수치는 작성 환경에서 **실제로 실행한 결과**입니다. 틈을 메운 정의는 H-10에 적었습니다.

요약

1. **틀:** 접두사 `N_` (분할) `K_` (전역 상수) `P_` (매개변수) `v_` (검증기, 참거짓으로 수축) `r_` (역방향 탐색).
2. **근거:** 구간 적분 결과는 유리수 계수 다항식이고 e, π는 초월수라서 조립제법의 나머지가 0이 되지 않는다. 그래서 계산은 전역 상수와 정확한 등식으로 만나지 못하고 부등식 구간에서만 만난다.
3. **패턴:** 오일러 패턴표의 다섯 종류(성장·감쇠, 회전, 주기, 다항식 배수, 기준 값)를 OSI·리눅스에서 관측되는 현상에 대응시켰다.
4. **다리:** ASCII(L6~L7) → C 구조체(L2~L4) → 컴파일러·어셈블리(엔디언, 다대일) → 격자점·점화식(L1 디지털 부분).
5. **틈 메우기:** 구조적 틈은 오일러 분해 요소(e, i, π, 0·1, t^k)로 정의해 대응시켰고(H-10), P≠NP로 가는 길은 대응 가정으로 H-11에 분리했다.

H-0. 구축 방향과 효과

구축 방향. 변수분리를 다음 순서로 세우면 OSI 7계층과 리눅스 명령이 하나의 틀로 묶입니다.

1. **이름 규칙을 고정합니다.** `lab.env` 한 곳에 전역 상수(`K_`)와 영역 이름(`N_`)을 `readonly`로 선언하고, 호출마다 달라지는 값은 `P_`로 받습니다.
2. **계층마다 검증기와 역탐색을 둡니다.** `v_l1` ~ `v_l7`이 참거짓 하나를 돌려주고, `r_trace`가 L1부터 돌려 첫 실패 계층을 원인 후보로 돌려줍니다.
3. **바이트를 바닥 표현으로 삼습니다.** L6~L7은 ASCII 코드표와 제어 코드, L2~L4는 C 구조체의 `sizeof`·`offsetof`·`_Static_assert`로 헤더 필드를 컴파일 타임 상수로 고정하고, 엔디언은 `htons`로 전선 순서와 호스트 순서를 분리합니다.
4. **패턴을 허용 구간으로 판정합니다.** 성장·감쇠, 회전, 주기, 다항식 배수, 기준 값 다섯 패턴마다 `K_`에 규약 값을 두고 `v_pat_*`로 구간 판정합니다.
5. **L1은 격자점과 점화식으로 구성합니다.** 심볼당 격자점을 정해 `for`로 구간을 평균하고 `if`로 문턱을 비교해 비트를 복원하며, CRC는 비트 단위 점화식으로 계산합니다.
6. **실행은 한 흐름입니다.** `source lab.env` → `P_` 지정 → `v_l1` … `v_l7` → `r_trace` → 원인 계층 보고.

```
layers.env   계층별 K_ / v_ / r_
ascii.env    L6~L7 바이트 층
pat.env      패턴 검증기
hdr.c        L2~L4 구조체 상수와 체크섬
l1.py        L1 격자점 모델, CRC 점화식
```

구축했을 때의 효과.

효과	구조에서 나오는 이유	실행으로 확인된 것
원인 계층을 바로 짚음	계층마다 검증기가 있고 `r_trace`가 L1부터 돌려 첫 실패에서 멈춤	닫힌 포트는 L4, 없는 인터페이스는 L1로 즉시 판정
오탐과 진짜 장애를 구분	도구가 없으면 "측정 불가"로 건너뜀	`ping`이 없는 환경에서 L3을 장애로 오판하지 않음
고칠 곳이 한 군데	IP, 포트, 이름, 경로를 `lab.env`의 전역 상수로 분리	값 14개를 한 파일에서 바꾸면 문서 전체 명령에 반영
실수로 값이 바뀌지 않음	`readonly` 상수	`K_L3_TTL=1`이 막히고 64 유지
호출이 서로 간섭하지 않음	매개변수는 복사본	함수 안에서 바꿔도 바깥 값은 그대로
노이즈에 흔들리지 않음	등식이 아니라 허용 구간으로 판정	백오프 `1.1 1.9 4.2 7.8` 통과, 비트오류율이 이론 구간과 일치
1바이트 변조도 탐지	체크섬·CRC가 기준 값에 닫힘	IP 헤더 접은 합 `0xffff`가 TTL 1바이트 변경으로 `0x1000`이 됨, CRC32도 불일치
어긋난 위치까지 지정	바이트 수준 역탐색 `r_firstdiff`	`cmp`가 어긋난 바이트를 "char 3"으로 보고
규약 위반을 빌드 때 차단	구조체 크기·오프셋을 `_Static_assert`로 고정	이더넷 14, IP 20, TCP 20과 필드 오프셋이 컴파일 시점에 검증
재현과 초기화가 쉬움	네임스페이스 랩과 일방향 장애 실습	명령 몇 줄로 생성, 두 줄로 삭제
자동화에 바로 연결	검증기가 종료 코드(0/1)를 돌려줌	cron, CI, 모니터링 스크립트에 그대로 연동
전 계층을 같은 방식으로 다룸	ASCII, 구조체, 격자점이 같은 접두사 규칙 아래	L1~L7을 `K_`·`P_`·`v_`·`r_` 하나의 틀로 점검

한 줄 요약: 상수와 매개변수를 나누고 검증기를 계층마다 세우면, 장애 지점을 빠르게 찾고 설정을 한 곳에서 관리하며 점검을 자동화할 수 있습니다.

H-1. 변수분리 다섯 원리와 이름 규칙

원리	뜻	접두사	리눅스에서의 모습
분할	영역은 서로소로 나뉜다	`N_`	네임스페이스, 프로세스·마운트 격리
전역 상수	값이 고정되고 안쪽에서 바꿀 수 없다	`K_`	`declare -rx`, 읽기 전용 마운트
매개변수	호출마다 달라지는 복사본	`P_`	함수 인자, 자식 프로세스가 물려받는 환경 변수
불리언 수축	결과를 참거짓 하나로 모아 원인 정보가 사라진다	`v_`	종료 코드 `$?`, `grep -q`, `nc -z`
역방향 탐색	수축된 결과에서 원인을 되짚는다	`r_`	`journalctl`, `tcpdump`, `cmp`, `strace`

H-2. 철학의 근거: 왜 계산과 전역 상수는 만나지 못하는가

1. 매개변수(격자 간격 h, 걸음 τ, 걸음 수, 계수, 요소 모양)는 정확 산술에서 **유리수**다.
2. 구간마다 적분한 결과는 **유리수 계수 다항식 p**다.
3. 전역 상수 e, π는 **초월수**(에르미트 1873, 린데만 1882)이므로 $p(e) \neq 0$, $p(\pi) \neq 0$이다. 조립제법의 나머지가 0이 되지 않는다.
4. 참 해의 적분 $(e^{\lambda\tau}-1)/\lambda$도 $\lambda\tau$가 0이 아닌 대수적 수이면 초월수(린데만–바이어슈트라스)라서 유리수와 같아질 수 없다.
5. 그래서 적분 항등식은 없고 **부등식 구간**만 남는다. 계산과 전역 상수는 비교의 자리에서만 만난다.

6. 항등식이 살아남는 곳은 한 바퀴를 온전히 도는 경우($\int_0^{2\pi} e^{it}dt = 0$, $e^{i\pi}+1 = 0$)뿐인데, τ가 유리수라 π의 정수배에 닿지 못한다. 오일러 항등식은 계산 안의 항등식이 아니라 **판정의 기준점**으로만 선다.

> **"만난다"의 정의를 앞에 둘 것.** 이 문서에서 "만난다"는 **정확한 등식으로 일치함**을 뜻합니다. 그 수준에서 위 정리가 불가능함을 증명하므로, 정의를 결론에 맞춰 고른 것이 아닙니다. 호출 지점의 값 전달(복사)은 이 정의의 만남이 아닙니다. 적용 범위는 상수 계수 선형계와 유리수 입력이며, 비선형계는 구간마다 야코비안을 얼린다는 가정(문서 1.7절)이 추가로 필요합니다.

찾은 곳	내용
검증기_철학_취합2 1절	상수는 계산에서 교집합이 없는 일방향 벡터. 일방향으로 흘러온 계산은 오일러 분해의 패턴에 대어 판정한다.
검증기_철학_취합2 2절	일방향 벡터 표, 조립제법 논증, 오일러 분해의 패턴표, 내적·외적(취합2에서 추가된 절)
검증기_철학_취합2 4절	확인은 앞으로 가서 닫히고 찾기는 거꾸로 가서 열려 있다. "문서의 명제가 아니라 저자의 입장."
검증기_철학_취합2 5절	"패턴 자체가 초월적이기 때문에 적분 항등식이 없다." 항등식이 사는 한 곳(한 바퀴)
본문 / 본문(작은거)	추출 텍스트가 완전히 동일. 본문에는 '일방향'과 '조립제법'이 0회 등장(철학 단락을 본문에 올리는 일이 남아 있음)
유지보수_261103_032620 (1)	텍스트가 추출되지 않아 읽지 못함(이미지로 보임)

H-3. '패턴'이라는 단어의 세 가지 쓰임

구분	출처	가리키는 것	성질
오일러 패턴	검증기_철학_취합(2) 1·2·5절	$t^k e^{\lambda t}$의 분해(성장·감쇠, 회전, 주기, 다항식 배수, 기준 값)	연속적·초월적
활성 패턴	본문 20.4 근처, 코드10 `pattern = []`	ReLU 뉴런의 켜짐/꺼짐 조합(은닉 폭 H이면 최대 2^H)	이산적·유한, 결정 열거 가능
디자인 패턴	세션_보안_파이프라인_전체	프론트 시간 이산화 값에서 읽는 선형 추세	관측된 경향

세 쓰임의 성질이 정반대라서 "패턴에 대어 판정한다"가 어느 쪽인지 독자가 가릴 수 없게 될 수 있습니다. 용어표를 두고 "오일러 패턴 / 활성 패턴 / 디자인 패턴"으로 접두어를 붙여 구분하세요. 이 부록의 H-4 이후에서 말하는 패턴은 **오일러 패턴**입니다.

H-4. 패턴 종류별 대응 (오일러 패턴표 기준)

패턴	OSI·리눅스에서 나타나는 곳	관측 명령	이상이면 의심
성장·감쇠	TCP 재전송 백오프, SYN 재전송(1·2·4·8초…), 혼잡 윈도우 초기 증가(RTT마다 2배), load average(1·5·15분 지수 감쇠 이동평균)	`ss -ti`, `tcpdump -ttt`, `nstat`, `uptime`	간격이 일정하면 백오프 미작동(L4), 감쇠 없이 상승하면 부하 누적
회전	ARP 이웃 상태 순환 (REACHABLE→STALE→DELAY→PROBE), TCP 상태 머신, 링크 플래핑	`ip neigh`, `ss -tan`, `carrier_changes`	한 상태에 고착(INCOMPLETE), 순환이 멈추지 않음(L1·L2)
주기·바퀴수	TCP keepalive(7200초 후 75초 간격 9회), DHCP 갱신(수명의 50%, 87.5%), TIME_WAIT 60초, ntp 폴링(64~1024초)	`sysctl net.ipv4.tcp_keepalive_*`, `journalctl`	주기·횟수가 틀어짐(설정 오류, 중간 장비, L5)
다항식 배수	TTL이 홉마다 1씩 감소, 시퀀스 번호·바이트 카운터의 선형 증가, DNS 캐시 TTL 카운트다운	`traceroute`, `ip -s link`, `dig`	기울기 급변, 카운터 정체(L2·L3)
기준 값	종료 코드 0, 체크섬 검증 시 정해진 값, ping 손실 0%, 인터페이스 UP	`$?`, `ping -c`, `ip -br link`	기준 값에서 벗어남(판정이 refuted인 지점)

주의: 오일러 패턴과의 대응은 **모양의 유비**입니다. "회전"은 복소수 회전이 아니라 상태가 순환한다는 뜻이고, TTL 감소는 지수가 아니라 선형(t^1)입니다. 혼잡 윈도우는 지수로 늘다가 임계점 이후 선형으로 바뀌어 한 현상 안에 두 패턴이 이어 붙습니다.

관측값에는 노이즈가 있으므로 패턴 검증기는 등식이 아니라 **허용 구간**으로 판정합니다. 지수 백오프 검증기(연속 간격의 비가 2±25% 안인가)의 실행 결과:

입력(초)	판정
1 2 4 8 16 32	O
1.1 1.9 4.2 7.8 (노이즈 포함)	O
1 1 1 1 (간격 일정)	X
1 2 4 4 (중간에 멈춤)	X
3 6 12	O

```
declare -rx K_PAT_BACKOFF_BASE=2 K_PAT_TOL=0.25
# 연속 간격의 비가 base±tol 구간 안인가 (등식이 아니라 구간으로 판정)
v_pat_backoff() {
  awk -v b="$K_PAT_BACKOFF_BASE" -v t="$K_PAT_TOL" 'BEGIN{ok=1}
    NR>1{ r=$1/prev; if (r<b*(1-t) || r>b*(1+t)) ok=0 } {prev=$1}
    END{exit !(ok && NR>1)}' <(printf '%s\n' "$@")
}
```

H-5. OSI 7계층 × 변수 접두사 (layers.env)

계층	전역 상수 `K_`	검증기 `v_`	매개변수 `P_`
L1	(없음)	`v_l1` 링크 UP	`P_IF`
L2	`K_L2_ETHERTYPE_ARP` (0x0806), `K_L2_MTU` (1500)	`v_l2` ARP 실패 없음	`P_IF`
L3	`K_L3_PROTO_ICMP` (1), `K_L3_TTL` (64)	`v_l3` ping	`P_TARGET`
L4	`K_L4_PROTO_TCP` (6), 포트 22·80·443	`v_l4` 포트 연결	`P_TARGET`, `P_PORT`
L5	`K_L5_KEEPALIVE_SEC` (7200)	`v_l5` keepalive 설정	(없음)
L6	`K_L6_CERT_WARN_SEC` (86400), `K_L6_CHARSET`	`v_l6` 인증서 만료	`P_HOST`, `P_PORT`
L7	`K_L7_HTTP_OK` (200), `K_L7_PORT_DNS` (53)	`v_l7` HTTP 200	`P_URL`

실행 결과 (루프백 환경):

상황	결과
정상 서비스	전 계층 정상 (이 환경에 ping이 없어 L3은 "측정 불가"로 건너뜀)
닫힌 포트	원인 후보: L4
없는 인터페이스	원인 후보: L1
상수 변경 시도 `K_L3_TTL=1`	`readonly variable` 오류, 값은 64로 유지
매개변수 변경	함수 안에서만 바뀌고 바깥 상수는 그대로(복사본)

수축으로 정보가 사라지는 일이 실제로 있었습니다. 처음에는 L3 검증기가 FAIL을 냈는데 원인은 망 장애가 아니라 `ping` 명령이 없었기 때문이었습니다. FAIL 하나로는 "도구 없음"과 "망 장애"를 구분할 수 없어서, 도구가 없으면 건너뛰도록 고쳤습니다.

```
#!/usr/bin/env bash
# layers.env — OSI 7계층 × 변수 분리
#   N_ = 분할(영역 이름)      K_ = 전역 상수(고정 규약, 안쪽에서 변경 불가)
#   P_ = 매개변수(호출마다 전달, 복사본)   v_ = 검증기(결과를 참/거짓 하나로 수축)
#   r_ = 역탐색(수축된 결과에서 원인 계층을 되짚음)

# ── N_: 분할 ──────────────────────────────────────────
declare -rx N_C=client N_S=server

# ── K_: 전역 상수 (계층별 고정 규약) ──────────────────
declare -rx K_L2_ETHERTYPE_ARP=0x0806 K_L2_MTU=1500
declare -rx K_L3_PROTO_ICMP=1 K_L3_TTL=64
declare -rx K_L4_PROTO_TCP=6 K_L4_PORT_SSH=22 K_L4_PORT_HTTP=80 K_L4_PORT_HTTPS=443
declare -rx K_L5_KEEPALIVE_SEC=7200
declare -rx K_L6_CERT_WARN_SEC=86400 K_L6_CHARSET=UTF-8
declare -rx K_L7_HTTP_OK=200 K_L7_PORT_DNS=53

# ── v_: 검증기 (순방향 확인. 반환값은 0/1 하나뿐 = 원인 정보가 사라짐) ──
v_l1() { local P_IF=$1;  [[ $(cat "/sys/class/net/$P_IF/operstate" 2>/dev/null) =~ ^(up|unknown)$ ]]; }
v_l2() { local P_IF=$1;  ! ip neigh show dev "$P_IF" 2>/dev/null | grep -qE 'FAILED|INCOMPLETE'; }
v_l3() { local P_TARGET=$1; ping -c1 -W1 "$P_TARGET" &>/dev/null; }
v_l4() { local P_TARGET=$1 P_PORT=$2; timeout 3 bash -c ">/dev/tcp/$P_TARGET/$P_PORT" 2>/dev/null; }
v_l5() { [[ $(sysctl -n net.ipv4.tcp_keepalive_time 2>/dev/null || echo 99999) -le $K_L5_KEEPALIVE_SEC
]]; }
v_l6() { local P_HOST=$1 P_PORT=$2
         echo | timeout 5 openssl s_client -connect "$P_HOST:$P_PORT" -servername "$P_HOST" 2>/dev/null \
           | openssl x509 -noout -checkend "$K_L6_CERT_WARN_SEC" &>/dev/null; }
```

```
v_l7() { local P_URL=$1; [[ $(curl -s -o /dev/null -m5 -w '%{http_code}' "$P_URL") == "$K_L7_HTTP_OK" ]];
}

# ── r_: 역탐색 (L1→L7 순서로 확인해 첫 실패 계층을 원인 후보로 반환) ──
r_trace() { local P_IF=$1 P_TARGET=$2 P_PORT=$3 P_URL=$4 L
  for L in 1 2 3 4 5 6 7; do
    case $L in
      1) v_l1 "$P_IF" ;;
      2) v_l2 "$P_IF" ;;
      3) command -v ping >/dev/null || { echo "L3 건너뜀: ping 없음(측정 불가 ≠ 장애)"; continue; }
         v_l3 "$P_TARGET" ;;
      4) v_l4 "$P_TARGET" "$P_PORT" ;;
      5) v_l5 ;;
      6) [[ $P_PORT != "$K_L4_PORT_HTTPS" ]] || v_l6 "$P_TARGET" "$P_PORT" ;;
      7) v_l7 "$P_URL" ;;
    esac || { echo "원인 후보: L$L"; return "$L"; }
  done
  echo "전 계층 정상"
}
```

H-6. ASCII 다리: L6~L7 텍스트 층

모든 계층의 헤더는 바이트이고, HTTP·SMTP·FTP는 ASCII 텍스트이며, 리눅스 파이프는 바이트를 흘립니다. 그래서 바이트·문자 수준에서 변수를 정의하면 OSI와 리눅스가 같은 변수로 묶입니다.

ASCII의 요소	변수분리의 역할	OSI에서의 자리	리눅스에서 보는 법
128개 코드표(7비트)	전역 상수 `K_ASCII_SIZE`	L6 표현	`man ascii`, `locale charmap`
문자·바이트 값	매개변수 `P_BYTE`	L7 페이로드	`od -c`, `tcpdump -A`
제어 코드 ACK(6), NAK(21), SYN(22), EOT(4)	프로토콜 신호 상수	초기 전송 프로토콜(BSC 등)의 제어, L2·L5의 확인·동기 개념	`cat -v`, `od -c`
패리티 비트	불리언 수축(1비트, 다대일)	L1·L2 오류 검출	`parity_even`
대소문자 차이 32, 숫자 '0'+n	선형 패턴(다항식 배수)	L7 텍스트 파싱	`tr 'a-z' 'A-Z'`
ROT13	회전·주기(두 번 적용하면 항등)	(유비)	`tr 'A-Za-z' 'N-ZA-Mn-za-m'`
CRLF(13,10)와 LF(10)	분할 경계의 불일치	L7 HTTP(CRLF) ↔ 리눅스(LF)	`cat -A`, `dos2unix`
7비트 밖(UTF-8 다바이트)	`v_ascii` 거짓	L6 인코딩	`iconv`, `file -i`
첫 번째로 어긋난 바이트	역방향 탐색 `r_firstdiff`	전 계층	`cmp`, `diff`, `od`

실행 결과: A=65, a=97(차이 32), Ctrl+A=0x01(= A & 31), `v_ascii 65` 참·`v_ascii 200` 거짓, 짝수 패리티 비트 A=0·C=1, ROT13 `Hello → Uryyb → Hello`, HTTP 요청의 CR·LF가 각 3개, `cmp` 가 `ABCD` 와 `ABXD` 의 어긋난 위치를 "char 3"으로 보고, 한(韓)은 `ed 95 9c` (3바이트, 모두 127 초과).

```
# K_: 전역 상수 (ASCII 규약)  P_: 매개변수(검사할 바이트/문자열)  v_: 검증기  r_: 역탐색
declare -rx K_ASCII_SIZE=128 K_ASCII_CASE_BIT=32 K_ASCII_CTRL_MASK=31
declare -rx K_ASCII_ACK=6 K_ASCII_NAK=21 K_ASCII_SYN=22 K_ASCII_EOT=4 K_ASCII_LF=10 K_ASCII_CR=13

v_ascii()  { local P_BYTE=$1; (( P_BYTE >= 0 && P_BYTE < K_ASCII_SIZE )); }           # 7비트 범위인가
parity_even() { local P_BYTE=$1 n=0 i; for ((i=0;i<7;i++)); do (( n += (P_BYTE>>i)&1 )); done; echo $((n%
2)); }
v_parity() { local P_BYTE=$1 P_BIT=$2; [[ $(parity_even "$P_BYTE") == "$P_BIT" ]]; }  # 1비트로 수축되는 검사
r_firstdiff() { local P_A=$1 P_B=$2; cmp "$P_A" "$P_B" 2>&1 | head -1; }              # 어긋난 첫 바이트 위치
```

H-7. 구조체·컴파일러·어셈블리 다리: L2~L4 바이너리 헤더

L1~L4의 헤더는 텍스트가 아니라 바이너리라서 ASCII로 닿지 못합니다. C 구조체와 컴파일러가 그 틈을 메웁니다.

변수분리	C 컴파일러 쪽	실행으로 확인된 것
전역 상수 `K_`	컴파일 타임 상수: `sizeof`, `offsetof`, `#define`, `_Static_assert`	이더넷 14, IP 20, TCP 20바이트와 필드 오프셋(EtherType 12, IP protocol 9, IP check 10, TCP window 14)이 모두 통과. 어긋나면 컴파일 자체가 실패
매개변수 `P_`	런타임에 구조체 필드에 채우는 값	MAC, IP(10.0.0.1→10.0.0.2), 포트(40000→8080), TTL 64, SYN 플래그
불리언 수축 `v_`	체크섬 검증	정상 헤더의 접은 합이 `0xffff`로 닫힘, TTL 1바이트만 바꾸면 `0x1000`으로 어긋남
역탐색 `r_`	어긋난 바이트 위치 찾기	`cmp`, `od`, `tcpdump -XX`

전선 위의 SYN 패킷 (54바이트)

```
ff ff ff ff ff ff 02 00 00 00 00 01 08 00          ← 이더넷(14) L2: 목적지·출발지 MAC, EtherType 0x0800
45 00 00 28 00 00 00 00 40 06 66 ce 0a 00 00 01 0a 00 00 02   ← IP(20) L3: 버전4·길이5, TTL 64, 프로토콜 6, 체크섬
9c 40 1f 90 00 00 00 00 00 00 00 00 50 02 ff ff 00 00 00 00   ← TCP(20) L4: 포트 40000→8080, 플래그 0x02(SYN)
```

엔디언과 어셈블리

전선에는 EtherType이 `08 00`으로 실리지만 리틀 엔디언 호스트가 그대로 읽으면 `0x0008`이고, `ntohs`를 거쳐야 `0x0800`이 됩니다. 이 변환은 어셈블리에서 `rolw $8, %ax`(16비트를 8칸 회전) 한 줄입니다. 바이트 순서 교환은 8비트 회전이고 두 번 적용하면 항등입니다(ROT13과 같은 구조). 네트워크 바이트 순서(전선)와 호스트 바이트 순서(메모리)를 분리해 두는 것이 변수분리의 가장 낮은 층 구현입니다.

컴파일은 일방향(다대일 수축)

`x*2`, `x+x`, `x<<1`은 소스가 서로 다르지만 **컴파일 결과가 3개 모두 같은 기계어**였습니다. 기계어에서 원래 소스를 정확히 복원할 수 없습니다(변수명, 주석, 쓰는 방식이 사라짐). 불리언 수축·노름과 같은 구조이며 "찾기는 열려 있다"는 문서의 문장과 맞습니다.

한계: 비트필드의 배치는 컴파일러와 아키텍처가 정합니다. 리눅스 헤더(`linux/ip.h`)가 `__LITTLE_ENDIAN_BITFIELD`와 `__BIG_ENDIAN_BITFIELD`로 구조체 정의를 둘로 나누는 것이 그 증거이며, 이식 가능한 코드는 비트필드 대신 시프트와 마스크를 씁니다. 규약의 일부를 컴파일러가 쥐고 있어서 완전히 닫히지는 않습니다.

```c
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include <arpa/inet.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>

/* K_: 컴파일 타임 상수. 규약과 어긋나면 컴파일 자체가 실패한다 */
_Static_assert(sizeof(struct ethhdr) == 14, "eth");
_Static_assert(offsetof(struct ethhdr, h_proto) == 12, "eth.proto");
_Static_assert(sizeof(struct iphdr) == 20, "ip");
_Static_assert(offsetof(struct iphdr, protocol) == 9, "ip.proto");
_Static_assert(offsetof(struct iphdr, check) == 10, "ip.check");
_Static_assert(sizeof(struct tcphdr) == 20, "tcp");
_Static_assert(offsetof(struct tcphdr, window) == 14, "tcp.win");

/* 빅엔디언으로 16비트 워드를 읽어 접는 합 (호스트 엔디언과 무관) */
```

```
static uint16_t fold(const void *p, size_t n) {
  const uint8_t *b = p; uint32_t s = 0;
  for (size_t i = 0; i + 1 < n; i += 2) s += (b[i] << 8) | b[i + 1];
  while (s >> 16) s = (s & 0xffff) + (s >> 16);
  return (uint16_t)s;
}

int main(void) {
  uint8_t pkt[sizeof(struct ethhdr) + sizeof(struct iphdr) + sizeof(struct tcphdr)];
  memset(pkt, 0, sizeof pkt);
  struct ethhdr *e = (void *)pkt; struct iphdr *ip = (void *)(e + 1); struct tcphdr *t = (void *)(ip +
1);

  /* P_: 런타임 매개변수 (호출마다 달라지는 값) */
  memcpy(e->h_dest, "\xff\xff\xff\xff\xff\xff", 6); memcpy(e->h_source, "\x02\x00\x00\x00\x00\x01", 6);
  e->h_proto = htons(ETH_P_IP);
  ip->version = 4; ip->ihl = 5; ip->ttl = 64; ip->protocol = IPPROTO_TCP;
  ip->tot_len = htons(sizeof *ip + sizeof *t);
  ip->saddr = htonl(0x0a000001); ip->daddr = htonl(0x0a000002);
  ip->check = 0; ip->check = htons((uint16_t)~fold(ip, sizeof *ip));
  t->source = htons(40000); t->dest = htons(8080); t->doff = 5; t->syn = 1; t->window = htons(65535);

  printf("[구조체] eth=%zu ip=%zu tcp=%zu  (모두 _Static_assert 통과)\n", sizeof(struct ethhdr), sizeof(stru
ct iphdr), sizeof(struct tcphdr));
  printf("[전선 순서] ");
  for (size_t i = 0; i < 14 + 20 + 20; i++) printf("%02x%s", pkt[i], (i == 13 || i == 33) ? " | " : " ");
  printf("\n[엔디언] 전선의 EtherType 바이트=%02x %02x, 호스트가 그대로 읽은 값=0x%04x, ntohs 후=0x%04x\n", pkt[1
2], pkt[13], e->h_proto, ntohs(e->h_proto));
  printf("[기준 값 닫힘] IP 헤더 접은 합(체크섬 포함)=0x%04x (0xffff면 정상)\n", fold(ip, sizeof *ip));
  printf("[플래그] TCP 13번째 바이트=0x%02x (SYN=0x02)\n", pkt[14 + 20 + 13]);
  ip->ttl ^= 0x10;
  printf("[1바이트 변조 후] 접은 합=0x%04x → %s\n", fold(ip, sizeof *ip), fold(ip, sizeof *ip) == 0xffff ? "정
상" : "불일치(검증기 FAIL)");
  return 0;
}
```

H-8. 격자점과 점화식: L1의 디지털 부분

L1의 연속 신호를 시간 격자점(한 심볼에 8점, 격자 간격 h = T/8)으로 쪼개면 수신기의 일이 그대로 `for`(구간 평균)와 `if`(문턱 비교)가 됩니다. 잡음 섞인 NRZ 파형으로 4만 비트를 복원한 결과, 시뮬레이션과 이론이 허용 구간 안에서 일치했습니다.

잡음 σ	시뮬레이션 비트오류율	이론값	구간 안
0.5	0.0000	0.0000	O
1.0	0.0024	0.0023	O
1.5	0.0285	0.0297	O
2.0	0.0778	0.0786	O

점화식은 L2에서도 이미 쓰입니다. 이더넷 FCS인 CRC32를 비트 단위 시프트 재귀로 계산한 값이 표준 확인값 `0xCBF43926`(입력 "123456789")과 일치했고 `zlib`과도 일치했습니다. 글자 하나를 바꾸면 `0xB2288182`로 달라져 검증기가 FAIL을 냅니다.

경계: 위 코드는 L1을 **모의한 것**이지 실제 케이블의 전압을 `for` 문이 움직이는 것은 아닙니다. 리눅스에서 실제 PHY에 닿는 명령은 `ethtool`(속도·듀플렉스·오토네고)과 드라이버가 노출한 레지스터 정도입니다. 또 아날로그→디지털 경계(표본화·양자화)가 곧 다대일 일방향 수축이며, 격자 근사의 오차는 구간 부등식으로 따로 가둬야 합니다.

```
import random, math, zlib
# K_: 전역 상수
K_SPS = 8                 # 한 심볼당 격자점 수 (격자 간격 h = T/SPS)
K_THR = 0.0               # 판정 문턱
K_CRC32_POLY = 0xEDB88320 # 이더넷 FCS 다항식(반사형)
K_TOL = 0.15              # 이론값과의 허용 상대 구간

def tx(bits):                         # 비트 -> 격자점 위의 파형 (NRZ: 1=+1, 0=-1)
    return [(1.0 if b else -1.0) for b in bits for _ in range(K_SPS)]

def rx(samples):                      # for/if 로 비트 복원 (구간 평균 후 문턱 비교)
    out = []
    for k in range(0, len(samples), K_SPS):
        m = sum(samples[k:k+K_SPS]) / K_SPS
        out.append(1 if m > K_THR else 0)
    return out

def ber(sigma, n=40000, seed=7):      # P_: 잡음 세기 sigma
    r = random.Random(seed)
    bits = [r.getrandbits(1) for _ in range(n)]
    wave = [v + r.gauss(0, sigma) for v in tx(bits)]
    got = rx(wave)
    return sum(a != b for a, b in zip(bits, got)) / n

def ber_theory(sigma):                # 구간 평균 뒤 잡음 표준편차 = sigma/sqrt(SPS)
    return 0.5 * math.erfc((1.0 / (sigma / math.sqrt(K_SPS))) / math.sqrt(2))

print("[L1 모의] 격자점+for/if 로 비트 복원, 시뮬레이션 vs 이론 (구간 판정)")
for s in (0.5, 1.0, 1.5, 2.0):
    a, b = ber(s), ber_theory(s)
    ok = abs(a - b) <= K_TOL * b + 3e-4
    print(f"  sigma={s:<3}  모의 BER={a:.4f}  이론={b:.4f}  구간 안={'O' if ok else 'X'}")

def crc32_bitwise(data):              # 점화식: 비트마다 if 로 갈라지는 시프트 재귀
    crc = 0xFFFFFFFF
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = (crc >> 1) ^ K_CRC32_POLY if crc & 1 else crc >> 1
```

```
    return crc ^ 0xFFFFFFFF

v = crc32_bitwise(b"123456789")
print(f"[L2 FCS] 점화식 CRC32('123456789')=0x{v:08X}  표준 확인값 0xCBF43926 일치={v==0xCBF43926}  zlib 일치={v
==zlib.crc32(b'123456789')}")
m = crc32_bitwise(b"123456780")
print(f"[1글자 변조] 0x{m:08X} → 불일치로 FAIL (다대일 수축 중 하나가 걸러짐)")
```

H-9. 층별 요약

층	다리	닿는 계층	상태
ASCII	코드표, 제어 문자, 텍스트 프로토콜	L6~L7	메워짐
C 구조체	헤더 필드 오프셋·크기·상수	L2~L4	메워짐(비트필드 순서는 컴파일러 의존)
컴파일러·어셈블리	컴파일 타임 상수와 런타임 값의 분리, 엔디언, 다대일 수축	구현 전반	메워짐
격자점·점화식	`for` / `if` 로 심볼 복원, CRC 재귀	L1 디지털, L2	메워짐(모의)
L5 세션	keepalive 설정 검사 정도	L5	회전(i)·주기(π) 요소로 정의(H-10)
L1 아날로그 매질	양자화·격자 오차의 상한 구간	L1	구간 부등식으로 가둠(H-10)

H-10. 오일러 분해 요소로 틈 메우기

문서의 오일러 분해 $t^k e^{\lambda t} = t^k e^{at}(\cos bt + i \sin bt)$와 $e^{i\pi}+1 = 0$의 요소(e: 길이·성장·감쇠, i: 회전, π: 주기·각도·바퀴 수, 0·1: 닫힘의 기준, t^k: 다항식 배수)로 OSI·리눅스에 남아 있던 틈을 **정의**했습니다. 정의는 판정보다 앞에서 선언하는 설계 결정입니다.

판정 형태의 이분법 (문서 5절을 그대로 옮김)

양쪽의 값	OSI·리눅스에서	판정
정확한 값 ↔ 정확한 값	바이트, 헤더 필드, 상태값, 포트, 체크섬	**정확한 등식** (`cmp` , `==` , 접은 합 `0xffff`)
정확한 값 ↔ 연속 참값	지연, 손실률, 재전송 간격, 신호 전압, 부하	**부등식 구간** (허용 오차 `K_TOL` , 오차 상한)

오일러 요소별 대응

틈	오일러 요소	메우는 정의
관측값 노이즈, L1 아날로그	e (길이, 나머지 사다리)	값은 구간 [L, U]로 가둔다. 양자화 오차 $\|x-q(x)\| \le \Delta/2$, 격자 간격 h의 사다리꼴 적분 오차 $\le (b-a)h^2/12 \cdot \max\|f''\|$. 상한이 없는 잡음은 확률 구간(비트오류율의 이론 구간)으로 가둔다.
데이터 경로의 양방향 (캡슐화 ↔ 역캡슐화)	한 바퀴 돌 때만 사는 항등식 ($\int_0^{2\pi} e^{it}dt = 0$, $e^{i\pi}+1 = 0$)	계층을 내려갔다 올라오는 한 바퀴는 정확한 등식(항등)으로 판정한다. 문서에서 항등식이 사는 한 곳과 같은 자리이고, 그 바깥의 측정값은 구간으로 판정한다.
엔디언, 비트필드 배치	π (반 바퀴)	바이트 순서 교환 `rolw $8` 은 16비트 고리의 반 바퀴이고 두 번 적용하면 항등이다. 비트 배치는 시프트와 마스크로 한 방향을 `K_` 상수로 고정한다.
L5 세션 (독립 계층 없음)	i (회전) + π (주기·바퀴 수)	`v_l5` := 세션 상태 순환(수립 → 유지 → 종료)이 `K_` 주기(keepalive 7200·75·9 등) 안에 한 바퀴를 닫는가.
교차 계층 (`iptables` , `conntrack` , MTU, DNS)	전역 상수 (모든 영역이 읽는 e·π·i)	여러 영역에 걸치는 규약은 `K_` 전역 상수로 올려 읽기 전용 교집합으로 둔다. 영역은 서로소, 교집합은 상수뿐이다.
명령 하나가 여러 계층을 봄	오일러 분해 (성분의 합)	한 관측을 성분으로 분해해 계층에 배분한다. 예: `curl -w` 의 `time_namelookup` (DNS), `time_connect` (L4), `time_appconnect` (L6 TLS), `time_starttransfer` (L7).
패턴 판정	e^{at}, i, π, t^k, 0·1	H-4의 다섯 패턴표 그대로 `v_pat_*` 로 구간 판정한다.

이 대응은 정의(설계 결정)입니다. 판정에 앞서 선언해 두면 결론에 맞춰 고른 것으로 읽히지 않습니다. P≠NP로 가는 연결은 여기서 정의한 대응이 아니라 H-11의 대응 가정으로 따로 둡니다.

H-11. P≠NP 서술의 위치와 확인이 필요한 간극

문서는 P vs NP를 "2026년 11월 현재 미해결"이라 쓰고 해당 절을 "문서의 명제가 아니라 저자의 입장"으로 두었습니다. 이 표기를 유지하는 것이 적절합니다. 이 부록의 대응으로 말할 수 있는 최대치는 다음 두 문장입니다.

> **구조에 대한 서술(사실).** 이 구조에서 실행 범위들은 서로소인 영역으로 분할된다. 전역 상수는 모든 영역이 읽을 수만 있는 값이고, 매개변수는 영역 사이를 한 방향으로만 건너는 값이다. 둘 모두 안쪽에서 바깥의 원본을 변경하는 경로가 없으므로, 이 구조의 값 이동은 쓰기 방향의 역연산을 갖지 않는다.
>
> **관점(은유).** 정보가 한 방향으로만 흐르고 참거짓으로 수축되는 구조에서는 순방향 확인과 역방향 복원이 대칭이 아니며, 이 비대칭은 P와 NP의 구분이 가진 구조와 닮아 있다. 본 부록은 이를 P≠NP의 구조적 은유로 제시한다.

간극	내용
1. 체(field)가 다름	MQ(다변수 이차 연립방정식)가 NP-완전인 것은 **유한체** 위에서다. 계산은 유리수·실수 위에서 일어나므로, 임의의 MQ 사례를 한 걸음의 역풀이로 심는 구성(환원)이 필요하다.
2. 최악 경우 vs 일방향 함수	NP-완전성은 **최악 경우**의 어려움이다. 일방향 함수는 평균적으로 거의 모든 입력에서 거꾸로 구하기 어려운 함수이며, 그 존재가 P≠NP를 함의한다(역은 알려져 있지 않음). MQ가 NP-완전하다는 것만으로는 일방향 함수의 존재가 따라 나오지 않는다.
3. 접근 불가 ≠ 계산 난이도	"되돌아가는 경로가 없다(쓰기 불가)"와 "되돌리는 데 시간이 많이 든다"는 다른 성질이다. 앞의 대응에서 '대응 가정'을 별도 문단으로 분리해 두어야 한다.
4. 독립변수(키)의 도입	키 같은 독립변수를 넣으면 키를 가진 쪽에는 역연산이 생긴다(함정문). "역연산이 없다"는 말은 관찰자가 가진 정보에 상대적인 성질로 고쳐 써야 하고, 암호는 P≠NP를 증명해 주는 것이 아니라 그 가정 위에 서 있다.

H-12. 남은 일

- ☐ "만난다 := 정확한 등식으로 일치함"을 일방향 벡터 정의의 첫 문장에 명시
- ☐ '일방향 벡터'와 조립제법 논증을 본문으로 올리기(본문에는 두 단어가 0회)
- ☐ '패턴' 용어 3종 구분표 작성(H-3)
- ☐ MQ 다리의 간극(체, 최악/평균)을 한 문단으로 명시(H-11)
- ☐ 패턴 검증기를 `tcpdump`·`ss` 출력과 연결(재전송 간격, ARP 상태 순환). 이번에는 랩 네임스페이스가 없어 단위 검증까지만 확인함
- ☐ L2~L4 헤더 상수(`K_ETH_*`, `K_IP_*`, `K_TCP_*`)를 `layers.env`에 통합
- ☐ 유지보수_261103_032620 (1).pdf는 텍스트가 없어 읽지 못함